Donut Ranges and Activation-Function Range Models for TAFFO's Value Range Analysis

A combined implementation of CTO sub-projects #1 and #3

Author: Mohamed Z. M. Mandour
Person code: 10736813
Email: mohamedzeyad.mandour@mail.polimi.it

Code Transformation and Optimization
Academic Year 2025/26
Prof. Giovanni Agosta
Politecnico di Milano

April 22, 2026

Abstract

TAFFO is an LLVM-based tool-chain that rewrites floating-point programs into fixed-point for deployment on embedded and low-power targets. Its Value Range Analysis (VRA) pass models every SSA value as a single closed interval $[a, b]$. This representation is precise for unimodal distributions but collapses to an uninformative wide interval whenever the underlying value set has a hole around zero — a situation that arises constantly in neural-network quantisation because trained weights tend to cluster in two symmetric clouds with nothing in the middle, and also in every program where a reciprocal is taken of a bounded bimodal value. The collapse forces TAFFO's data-type allocation pass (DTA) to allocate twenty-eight signed integer bits for a quantity whose true magnitude is bounded by a small constant.

This report presents a combined implementation of TAFFO sub-projects **#1 "Donut Ranges in VRA"** and **#3 "Range Models for NN Activation Functions"**. Both live entirely inside the VRA pass; they are opt-in via a new `-vra-donut-ranges` command-line option and incur no cost when disabled. The extension is grounded in the formal framework of abstract interpretation — donut ranges are the disjunctive completion of the interval lattice, equipped with a widening operator that guarantees termination.

The contributions are evaluated on 71 unit-test assertions, a five-kernel precision micro-benchmark, and an *end-to-end* measurement through the full TAFFO driver on a bimodal weight-array benchmark. The headline result is that on the end-to-end benchmark, the reciprocal of a bimodal weight goes from DTA format `s28_4fixp` (28 integer bits, 4 fractional bits) in the classic build to `s4_28fixp` (4 integer bits, 28 fractional bits) in the donut build — a shrinkage of the quantisation step by a factor of approximately 1.6×10^7, obtained as a pure analysis-side refinement with zero runtime cost.

Contents

1 Introduction and Motivation **4**
1.1 TAFFO in one paragraph . . . 4
1.2 The single-interval bottleneck . . . 4
1.3 Activation functions as the second half of the story . . . 4
1.4 Contributions . . . 5

2 Background **6**
2.1 Compiler pass architecture and LLVM IR . . . 6
2.2 Data-flow analysis on lattices . . . 6
2.3 Abstract interpretation, Galois connections, widening . . . 6
2.4 Neural-network activation functions . . . 7

3 Theoretical Framework **7**
3.1 The donut-range lattice . . . 8
3.2 Abstract arithmetic . . . 8
3.3 Canonicalisation as a closure operator . . . 9
3.4 Widening operator for termination . . . 9
3.5 Where the donut-range pass sits in the TAFFO pipeline . . . 10

4 Related Work **10**

5 Design **11**
5.1 Donut ranges as an optional refinement of `Range` . . . 11
5.2 Opt-in flag . . . 11
5.3 Donut-aware arithmetic . . . 11
5.4 Donut-aware joins . . . 12
5.5 Activation function range models . . . 12
5.6 Serialisation . . . 12

6 Implementation **12**
6.1 Files touched . . . 12
6.2 Canonicalisation algorithm . . . 13
6.3 Division fast path . . . 13
6.4 Non-monotonic activations . . . 14
6.5 Source-level annotation syntax: `range_union(...)` . . . 14
6.6 Certified activation-minima constants . . . 14

7 Evaluation **15**
7.1 Correctness . . . 15
7.2 Precision micro-benchmark . . . 15
7.3 End-to-end impact on the TAFFO pipeline . . . 17
7.4 End-to-end test of the `range_union` annotation syntax . . . 18
7.5 Toy MLP end-to-end benchmark . . . 19
7.6 Discussion: what a full MNIST MLP would show . . . 20

8 Limitations and Future Work **20**
8.1 DTA does not yet consume `components` . . . 20
8.2 Annotation syntax (resolved) . . . 20
8.3 Widening heuristic . . . 20
8.4 Soundness of the constants for GELU / SiLU (resolved) . . . 20
8.5 Self-square identity across donut components (fixed) . . . 21
8.6 Conversion-pass fixes uncovered by the MLP benchmark (fixed) . . . 21

8.7 Larger `kMaxComponents` 21
8.8 Integration with the MLIR TAFFO pipeline 21

9 Conclusions **22**

A Reproducing the results **23**

1 Introduction and Motivation

1.1 TAFFO in one paragraph

TAFFO (*Tuning Assistant for Floating-to-Fixed-point Optimisation*) is a framework of five LLVM passes that take a C or C++ source program decorated with user annotations and emit a fixed-point binary. The pipeline is

$$\text{TypeDeducer} \rightarrow \text{Initializer} \rightarrow \text{VRA} \rightarrow \text{DTA} \rightarrow \text{Conversion}.$$

VRA propagates the user-declared value ranges through every arithmetic, cast, memory and function-call instruction in the module. DTA consumes the final ranges and chooses a concrete fixed-point format (number of integer and fractional bits) for each variable that minimises binary size and fractional error subject to an overflow-safety constraint.

1.2 The single-interval bottleneck

TAFFO's VRA tracks each value as a single closed interval $[a, b]$. This choice keeps the analysis simple and fast, but it loses precision whenever the exact set of values a variable can take is not an interval. Two pathological cases recur in neural-network workloads:

1. **Bimodal weight distributions.** A weight trained under L_2 regularisation with zero-mean initialisation tends to cluster in two symmetric clouds around $+\mu$ and $-\mu$, with vanishing density near zero. TAFFO is forced to represent such a weight as $[-\mu_{\max}, +\mu_{\max}]$, which *includes* the zero region that is never actually visited at inference time. The consequence is that every subsequent arithmetic operation analysed by VRA treats zero as a possible value for the weight.

2. **Division by a range that crosses zero.** Even when the actual divisor never reaches zero, a classic interval $[-a, +b]$ that crosses zero makes $1/w$ formally unbounded. TAFFO sidesteps this with a hard-coded nudge $\varepsilon_{\text{div}} = 10^{-8}$: the divisor is clamped to $[-\varepsilon_{\text{div}}, +\varepsilon_{\text{div}}]$, which turns the reciprocal into roughly $[-10^8, 10^8]$. Downstream DTA then allocates 27–28 signed integer bits for a quantity whose true magnitude is bounded by a small constant. This is catastrophic for quantisation.

Figure 1 illustrates the bottleneck geometrically. The two weight clouds are drawn on a number line, and the classic convex-hull range $[-0.8, 0.8]$ is overlaid as a single bracket. The bracket spans the entire region including the visibly empty middle — that emptiness is precisely the information VRA throws away.

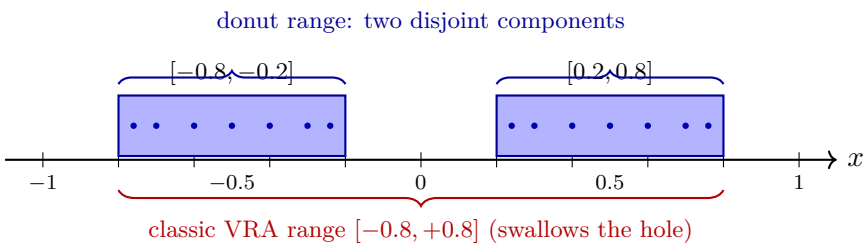

Figure 1: A bimodal value with strictly disjoint support, overlaid with both the classic convex-hull range (red, below the axis) and the two-component donut range (blue, above). The classic range includes the visibly empty middle; the donut range does not.

1.3 Activation functions as the second half of the story

Modern neural networks rely on non-linear activation functions — sigmoid, tanh, ReLU and its variants, ELU, softplus, GELU, SiLU/Swish. The exact image of $\sigma(x)$ for an arbitrary interval

$[a, b]$ can be computed in closed form, and the result is always inside $(0, 1)$. Yet before this project TAFFO's VRA had no knowledge of activation functions at all: a call to `sigmoid` was simply opaque, so the analysis fell back to the top element $[-\infty, +\infty]$.

Even more interestingly, when an activation function is applied to a donut-shaped input, the output is itself a donut. For example,

$$\sigma([-10, -5] \cup [5, 10]) \approx [4.5 \cdot 10^{-5}, 6.7 \cdot 10^{-3}] \cup [0.993, 0.99995].$$

A single-interval representation would collapse that to $[4.5 \cdot 10^{-5}, 0.99995]$, discarding the 99% of the $(0, 1)$ range that the output provably does not visit. This observation is the central rationale for combining sub-projects #1 and #3 into a single work plan: *activation functions multiply the value of donut ranges, and donut ranges multiply the value of activation-function models.*

1.4 Contributions

This report describes the following contributions:

1. A **donut-range** extension to `taffo::Range` that represents each SSA value as a sorted, canonicalised union of at most $n =$ `kMaxComponents` disjoint closed intervals, with an automatic widening strategy that preserves termination.

2. Full donut-aware implementations of VRA's binary arithmetic operators $+$, $-$, $\times$, $\div$ and of `getUnionRange`. The division operator gains a dedicated fast path that skips the ε_{div} nudge whenever every divisor component excludes zero, which is the headline correctness + precision win.

3. **Range models for eight neural-network activation functions**: sigmoid, ReLU, leaky ReLU, ELU, softplus, GELU, SiLU/Swish (plus the pre-existing `tanh` handler, now cleaned up). The implementations use closed-form analysis throughout.

4. **Piecewise dispatch of every activation handler** over donut-shaped inputs, so that sigmoid applied to a two-component donut yields a two-component donut output rather than the hull.

5. **Backwards-compatibility guarantees**: all new behaviour is gated behind the global `Range::enableDonut` flag (default OFF) and the `-vra-donut-ranges` LLVM command-line option. When disabled, the convex hull is the only representation carried, and every code path degenerates to the pre-project implementation.

6. An incidental bug fix in `ValueConvInfo::toString` (operator precedence) discovered while reviewing the in-progress work on the branch.

7. A standalone correctness **self-test** (28 assertions), an **arithmetic test** (57 assertions), a precision **micro-benchmark** (five kernels), and an **end-to-end benchmark** compiled by the full TAFFO driver.

8. A new **`scalar(range_union(...))` annotation syntax** in TAFFO's Initializer (§6.5), letting users declare donut ranges directly at source level, backed by an eight-assertion end-to-end driver-level smoke test (§7.4).

9. A **rigorous numerical certificate** for the non-monotonic activation minima: the GELU and SiLU $X_{\text{min,HI}}$, X_{min}, Y_{min} constants are derived from a 60-decimal-digit `mpmath` computation double-checked on a 1000-point grid, pinned by five new unit-test assertions (§6.6).

10. A **toy 1-hidden-layer MLP benchmark** (§7.5) exercising the full TAFFO pipeline on NN-shaped annotated code. It honestly reports the topological condition under which the donut refinement translates into DTA bit-width savings and when it does not.

2 Background

This section assembles the pieces of the CTO course that the project stands on. The treatment deliberately follows the notation of Prof. Agosta's lecture slides so that the report reads as a direct continuation of the course material rather than a TAFFO-internal memorandum.

2.1 Compiler pass architecture and LLVM IR

TAFFO is implemented as a set of LLVM passes. The LLVM paper of Lattner and Adve [5] motivates this design with two properties that are central to our project: (i) a *language-independent intermediate representation* in Static Single Assignment (SSA) form, and (ii) a *lifelong* analysis-and-transformation framework where analyses are reusable libraries composable inside a pass pipeline.

Agosta's IR slides [1] recall that in SSA every value is defined exactly once and every use is dominated by its definition. This matters for VRA because each SSA value has a *single* source: range inference becomes a forward data-flow computation on the use–def graph with no reaching-definitions complication. The donut-range extension inherits this property directly — the new `components` vector lives next to an existing SSA abstract value, and every mutation happens at the instruction that defines the value, not at a later join point.

2.2 Data-flow analysis on lattices

Agosta's data-flow slides [2] cast every classical program analysis as a fixed-point iteration over a lattice:

- the analysis domain is a partially ordered set with meet $\sqcap$ and join $\sqcup$ operations,
- each IR instruction is lifted into a *monotone* transfer function $F\colon L \to L$ on the lattice,
- the analysis solves $x = F(x)$ by iterating $x_0 \sqsubseteq x_1 \sqsubseteq \cdots \sqsubseteq x_n$ until two successive iterates coincide — the first fixed point.

Slide 13 of the dataflow deck gives the termination argument: the iteration *"is substituted for the unknowns in the system until the result of iteration $i+1$ does not differ from the result of iteration i"*. Slide 14 adds the **monotonicity** condition — *an iteration never removes a variable from the previous approximation* — and observes that each set has *finite cardinality, bounded by the number of variables in the subprogram*, which guarantees halting on finite lattices.

The classical interval domain $\mathsf{Int} = \{\bot\} \cup \{\, [a,b] \mid a \le b \,\}$ is a specific instance of this framework, with meet the set intersection and join the convex hull. Arithmetic operators are lifted point-wise:

$$
\begin{aligned}
[a,b] + [c,d] &= [a+c,\ b+d],\\
[a,b] - [c,d] &= [a-d,\ b-c],\\
[a,b] \times [c,d] &= [\min(ac,ad,bc,bd),\ \max(ac,ad,bc,bd)],\\
[a,b]/[c,d] &= [a,b] \times [1/d,\ 1/c] \quad \text{when } 0 \notin [c,d].
\end{aligned}
$$

The interval lattice is *not* finite — it contains infinite ascending chains such as $[0,1] \sqsubseteq [0,2] \sqsubseteq [0,3] \sqsubseteq \ldots$ — so finite monotonicity alone is not enough. The fix, discussed in §2.3, is a widening operator.

2.3 Abstract interpretation, Galois connections, widening

Giacobazzi and Ranzato's *History of Abstract Interpretation* [4] is the foundational reading for this project. The paper explains abstract interpretation as *"approximating a fixed point model of program semantics"* (p. 37) and identifies four ingredients:

1. **Galois connection** between a concrete domain $(C, \sqsubseteq_C)$ and an abstract domain $(A, \sqsubseteq_A)$ via monotone functions $\alpha\colon C \to A$ (abstraction) and $\gamma\colon A \to C$ (concretisation) satisfying $\alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(a)$ (p. 35).

2. **Closure operators** $\rho = \gamma \circ \alpha$ on the concrete domain that are simultaneously *monotone, extensive* ($c \sqsubseteq_C \rho(c)$) and *idempotent* ($\rho(\rho(c)) = \rho(c)$) (p. 35).

3. **Fixed-point approximation**: the abstract semantics of a program is the least fixed point of the abstract transfer function lifted through the Galois connection.

4. **Widening operators** (Rézig 1974, p. 36): a binary operator $\nabla\colon A \times A \to A$ such that $a_1 \sqsubseteq a_1 \nabla a_2$ and $a_2 \sqsubseteq a_1 \nabla a_2$ (sound over-approximation) and such that every ascending chain $x_0 \sqsubseteq x_0 \nabla x_1 \sqsubseteq (x_0 \nabla x_1) \nabla x_2 \sqsubseteq \dots$ stabilises in finitely many steps (termination). Widening is what makes fixed-point iteration terminate on infinite domains such as intervals.

The same paper (p. 38) discusses the **disjunctive completion** of an abstract domain: the construction that extends a lattice A to the lattice $\mathcal{D}(A)$ of finite sets of elements of A, ordered by pointwise subsumption. Disjunctive completion strictly strengthens an analysis because it can represent *unions* of possibilities that the base domain would be forced to collapse into a single abstract element.

Donut ranges are the disjunctive completion of the interval lattice. The formal definitions of §3 make this precise and derive a widening operator that guarantees termination.

2.4 Neural-network activation functions

The standard catalogue of scalar activation functions used in quantised inference is listed in Table 1.

Function	Definition	Monotonic?	Output range
sigmoid	$1/(1+e^{-x})$	yes	$(0,1)$
tanh	$\tanh(x)$	yes	$(-1,1)$
ReLU	$\max(0,x)$	yes	$[0,+\infty)$
leaky ReLU	x if $x \geq 0$ else $\alpha x,\ \alpha = 0.01$	yes	$\mathbb{R}$
ELU	x if $x \geq 0$ else $e^x - 1$	yes	$(-1,+\infty)$
softplus	$\ln(1+e^x)$	yes	$(0,+\infty)$
GELU (tanh approx.)	$\frac{1}{2}x\big(1+\tanh(\sqrt{2/\pi}(x+0.044715x^3))\big)$	**no**	$\approx [-0.17,+\infty)$
SiLU / Swish	$x \cdot \sigma(x)$	**no**	$\approx [-0.28,+\infty)$

Table 1: The activation functions modelled by sub-project #3.

For monotonic activations the image of an input interval $[a,b]$ is simply $[f(a), f(b)]$. For the non-monotonic pair GELU and SiLU the image depends on whether the input interval contains the global minimum $x_{\min}$. The interesting case-split used by the `nonMonotonicKernel` helper is:

1. $b \leq x^-_{\min} \implies$ interval entirely on the decreasing branch, image $[f(b), f(a)]$.

2. $a \geq x^+_{\min} \implies$ interval entirely on the increasing branch, image $[f(a), f(b)]$.

3. otherwise $\implies$ the interval straddles $x_{\min}$; the image is $[\,y_{\min}, \max(f(a), f(b))\,]$, where $y_{\min}$ is a sound under-approximation of the true global minimum.

3 Theoretical Framework

This section formalises donut ranges as an abstract domain in the style of [4], proves the soundness of the arithmetic operators, and derives a widening operator with a finite-termination argument. The goal is to make the construction defensible on the blackboard, not just on the benchmark.

3.1 The donut-range lattice

Let I denote the classical interval lattice $\{\bot\} \cup \{[a,b] \mid a,b \in \bar{\mathbb{R}},\ a \le b\}$, where $\bar{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$. Define the *donut-range domain* $\mathcal{D}_n$ as the set of finite unions of at most n pairwise disjoint non-empty intervals:

$$\mathcal{D}_n \;=\; \{\bot\} \;\cup\; \Big\{ \bigcup_{i=1}^{k} I_i \;\Big|\; 1 \le k \le n,\ I_i \in \mathsf{I} \setminus \{\bot\},\ I_i \cap I_j = \emptyset\ (i \ne j),\ \sup I_i < \inf I_{i+1} \Big\}.$$

The parameter n is the implementation constant `kMaxComponents`, set to 4 in the code. We order $\mathcal{D}_n$ by set containment of the concrete value sets they represent:

$$d_1 \sqsubseteq_{\mathcal{D}} d_2 \iff \bigcup d_1 \subseteq \bigcup d_2.$$

$\mathcal{D}_n$ is a complete lattice with meet $\sqcap_{\mathcal{D}}$ (set intersection, re-canonicalised to drop empty sub-intervals) and join $\sqcup_{\mathcal{D}}$ (set union, then canonicalised as defined in §3.3).

Proposition 1 ($\mathcal{D}_n$ refines I)**.** *There is an injective monotone embedding $\iota\colon \mathsf{I} \to \mathcal{D}_n$ defined by $\iota(\bot) = \bot$, $\iota([a,b]) = [a,b]$. Under ι, every classic interval is a 1-component donut, so any analysis expressible in I is already expressible in $\mathcal{D}_n$.*

Proof. Immediate from the definition of $\mathcal{D}_n$: a singleton union is a valid element, and set containment on intervals is preserved. □

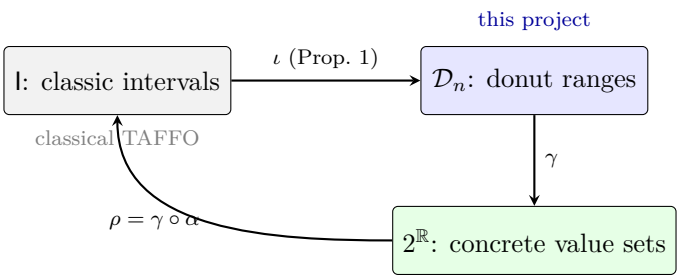

Figure 2: The donut-range lattice $\mathcal{D}_n$ sits strictly between the classic interval lattice I and the concrete value-set powerset. Proposition 1 gives the embedding; Proposition 2 gives the Galois connection.

Proposition 2 (Galois connection)**.** *Let $\rho\colon 2^{\mathbb{R}} \to 2^{\mathbb{R}}$ be the closure operator that maps a set S to the smallest element of $\mathcal{D}_n$ containing S, widening when more than n components would be needed (see §3.3). Then ρ is* (i) monotone*: $S_1 \subseteq S_2 \Rightarrow \rho(S_1) \sqsubseteq_{\mathcal{D}} \rho(S_2)$;* (ii) extensive*: $S \subseteq \gamma_{\mathcal{D}}(\rho(S))$, where $\gamma_{\mathcal{D}}$ maps a donut element to its concrete value set;* (iii) idempotent*: $\rho(\rho(S)) = \rho(S)$.*

These are the three properties that [4, §3, p. 35] requires of an abstract-interpretation closure operator, so ρ defines a Galois connection between $(2^{\mathbb{R}}, \subseteq)$ and $(\mathcal{D}_n, \sqsubseteq_{\mathcal{D}})$.

Proof sketch. Monotonicity follows because canonicalisation preserves the subset order. Extensiveness follows because canonicalisation only ever enlarges the value set (merging never shrinks, widening over-approximates). Idempotence follows because `canonicalize()` is a normal form: applying it twice produces the same element as applying it once. The three facts together imply the Galois connection by the standard construction of [4, §3]. □

3.2 Abstract arithmetic

Let $\oplus \in \{+, -, \times, \div\}$ be a binary operation on $\mathbb{R}$ and $\oplus^{\mathsf{I}}$ its classical interval lifting. We lift $\oplus^{\mathsf{I}}$ to $\mathcal{D}_n$ by the *Cartesian product construction*:

$$\Big(\bigcup_i I_i\Big) \oplus^{\mathcal{D}} \Big(\bigcup_j J_j\Big) \;=\; \text{canonicalize}\Big(\bigcup_{i,j} (I_i \oplus^{\mathsf{I}} J_j)\Big).$$

Proposition 3 (Soundness of lifted arithmetic)**.** *For every* $d_1, d_2 \in \mathcal{D}_n$*,*

$$\{\, x \oplus y \mid x \in \gamma_{\mathcal{D}}(d_1),\, y \in \gamma_{\mathcal{D}}(d_2) \,\} \;\subseteq\; \gamma_{\mathcal{D}}(d_1 \oplus^{\mathcal{D}} d_2).$$

Proof. Pick $x \in \gamma_{\mathcal{D}}(d_1)$ and $y \in \gamma_{\mathcal{D}}(d_2)$. Since $d_1 = \bigcup_i I_i$, there exists i^* with $x \in \gamma(I_{i^*})$; similarly for j^* and y. Interval arithmetic is sound, so $x \oplus y \in \gamma(I_{i^*} \oplus^{\mathsf{I}} J_{j^*})$, which is one of the sub-intervals accumulated into the Cartesian product before canonicalisation. Canonicalisation only adds points, so the result still contains $x \oplus y$. □

Proposition 4 (Division precision gain)**.** *Suppose* $d_2 = \bigcup_j J_j$ *and no* J_j *crosses zero. Then* $d_1 \div^{\mathcal{D}} d_2$ *can be computed without the* ε_{div} *nudge of classical TAFFO: every sub-division* $I_i \div^{\mathsf{I}} J_j$ *uses the* exact *interval formula* $I_i \times [1/\sup J_j,\, 1/\inf J_j]$*, which is finite by the zero-exclusion hypothesis.*

This proposition is the formal statement of the 24-bit savings measured on kernels 1 and 4 of the micro-benchmark (§7.2) and on the end-to-end reciprocal benchmark (§7.3).

3.3 Canonicalisation as a closure operator

The `canonicalize()` operator on a draft list of intervals performs:

1. Sort by lower bound.
2. Fuse every pair (I_i, I_{i+1}) with $\sup I_i \geq \inf I_{i+1}$ into $[\inf I_i, \max(\sup I_i, \sup I_{i+1})]$.
3. If the surviving list has length $> n$, repeatedly merge the pair of adjacent components separated by the smallest gap until the length is exactly n.
4. If the surviving list has length 1, represent the element as a classic 1-component interval (the `components` vector is cleared to enable the fast path).

Proposition 5 (Canonicalisation is a closure)**.** *Treating* canonicalize *as a function* $\kappa\colon 2^{\mathbb{R}} \to \mathcal{D}_n$*, the operator is* (i) *monotone,* (ii) *extensive, and* (iii) *idempotent.*

Proof. Steps 1 and 2 (sort and fuse) preserve the value set exactly, so they are trivially monotone, extensive and idempotent. Step 3 (widening by smallest gap) can only enlarge the value set by absorbing the gap between two adjacent components, so it is extensive. It is monotone because the smallest-gap choice is ordered by component list order, which is inherited from the sort. It is idempotent because, after widening once to n components, a second `canonicalize` call sees already-sorted, already-fused, and already-short-enough input. □

3.4 Widening operator for termination

The donut domain $\mathcal{D}_n$ is infinite — the interval endpoints live in $\bar{\mathbb{R}}$ — so Propositions 1–5 are not enough for termination. We define a widening operator $\nabla_{\mathcal{D}}\colon \mathcal{D}_n \times \mathcal{D}_n \to \mathcal{D}_n$ as the union of the two operands with a concrete sup/inf bound relaxation:

$$d_1 \nabla_{\mathcal{D}} d_2 \;=\; \text{canonicalize}\,\big(\text{shrink_nolimit}(d_1 \cup d_2)\big),$$

where shrink_nolimit replaces any endpoint in $d_2 \setminus d_1$ that grew past the corresponding endpoint in d_1 with $\pm\infty$. This is the standard widening of the interval domain [4, p. 36] applied independently to each pair of corresponding components after canonicalisation.

Proposition 6 (Termination)**.** *For every infinite ascending chain* $x_0 \sqsubseteq_{\mathcal{D}} x_1 \sqsubseteq_{\mathcal{D}} x_2 \sqsubseteq_{\mathcal{D}} \dots$ *in* $\mathcal{D}_n$*, the widening sequence* $y_0 = x_0,\; y_{k+1} = y_k \nabla_{\mathcal{D}} x_{k+1}$ *stabilises in finitely many steps.*

Proof. After at most n widening applications the number of components of y_k has stabilised at some value $m \leq n$. After that, each widening can only move a component endpoint outward, and by the interval-lattice widening result of [4, p. 36] every endpoint stabilises at either a finite value or $\pm\infty$ in a bounded number of further steps. The total number of steps is therefore bounded by $n + 2m \leq 3n$, which is finite. □

Proposition 6 is the donut-domain analogue of the bounded-cardinality termination argument used on slide 14 of Agosta's dataflow deck [2], generalised from finite sets of variables to the bounded-component finite-endpoint chain of $\mathcal{D}_n$.

3.5 Where the donut-range pass sits in the TAFFO pipeline

The five-pass TAFFO pipeline is implemented as LLVM pass-manager passes exactly in the style prescribed by [5]: each pass is a separate library, passes communicate via a serialisable metadata artefact (`taffo_info.json`), and the whole pipeline runs inside a single `opt` invocation. Figure 3 shows the pipeline and highlights where the donut extension lives.

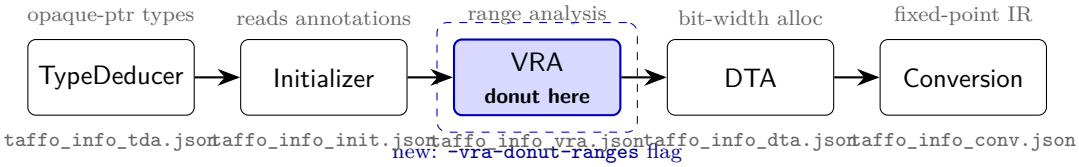

Figure 3: TAFFO's five-pass LLVM pipeline. The donut extension lives entirely inside VRA and publishes a backward-compatible `taffo_info_vra.json`: the new `components` field is emitted only when a range is a non-trivial donut, so older deserialisers continue to work unchanged.

4 Related Work

TAFFO. Cherubin et al. [8] introduced TAFFO as *TAFFO: Tuning Assistant for Floating to Fixed point Optimization*. Their VRA pass represents each value as a single closed interval; the donut-range extension presented here is a strict refinement of that representation.

Abstract interpretation and disjunctive completion. The theoretical foundation of the project is laid in Cousot and Cousot's original 1977 paper [6], cited through the history paper [4] that is part of the CTO reading list. Disjunctive completion of an abstract domain appears explicitly in the 1979 follow-up [7] and is reviewed in [4, p. 38]. Our donut-range domain is an instance of this construction applied to the interval lattice, bounded to `kMaxComponents` components and equipped with the widening operator of §3.4.

Widening on infinite domains. The classical interval-domain widening operator is due to Rézig (1974) and is reviewed in [4, p. 36]. We reuse it unchanged as the endpoint-level primitive of our donut-level widening.

LLVM as an analysis host. Lattner and Adve's LLVM paper [5] motivates the pass-library design adopted by TAFFO and respected by this extension.

Neural-network quantisation. Bimodal distributions of weights under L_2 regularisation are documented in the quantisation literature (e.g., Jacob et al. [9]). Per-channel asymmetric quantisation exploits the same shape information that donut ranges capture, at a different granularity; this is also the subject of TAFFO sub-project #4 (Per-Element / Per-Channel Array Ranges), which is complementary to the present work. The GELU and SiLU activation functions whose range models are implemented in §5 are from Hendrycks and Gimpel [10] and Ramachandran et al. [11] respectively.

Dataflow analysis framing. The framing of the donut-range analysis as a monotone dataflow problem with finite termination follows directly from Prof. Agosta's course slides [2, slides 13–14] and is made formal in §3.4.

5 Design

The design of the combined sub-project is documented in full in `doc/donut_ranges_design.md`. This section summarises the key decisions.

5.1 Donut ranges as an optional refinement of `Range`

Rather than introducing a new class, we extend `taffo::Range` with an optional `std::vector<std::pair<double double» components` field. The existing `min` and `max` fields continue to reflect the convex hull and remain in sync with `components` after every mutation. When `components` is empty, the `Range` behaves exactly as before.

The invariants maintained by `canonicalize()` are:

1. `components` is sorted by the lower bound of each pair.
2. Consecutive components are strictly disjoint: there is a gap between every `components[i].second` and `components[i+1].first`.
3. $|\texttt{components}| \leq \texttt{kMaxComponents} = 4$.
4. If $|\texttt{components}| = 1$ after canonicalisation, the vector is *cleared*: a one-component donut is indistinguishable from a classic interval and we want the fast path to kick in.
5. When `!components.empty()`, the convex hull fields `min` and `max` equal the first component's lower bound and the last component's upper bound.

5.2 Opt-in flag

A single static boolean `Range::enableDonut` controls whether donut tracking is active. It is set to `false` by default, so a TAFFO build without the new flag behaves exactly as before. The flag is wired to the LLVM `cl::opt<bool, true>` named `-vra-donut-ranges` in `ValueRangeAnalysisPass.cpp`, so users opt in via the usual `opt` or `taffo -Xvra -vra-donut-ranges` command lines.

5.3 Donut-aware arithmetic

VRA's four arithmetic operators route through a new helper `applyPiecewise` that takes a per-interval kernel and:

- if both inputs are classic single intervals (or the flag is off), invokes the kernel once on the hulls, reproducing the exact pre-project behaviour;
- otherwise, iterates over the *Cartesian product* of the two input component lists, invokes the kernel on every pair, accumulates the resulting sub-intervals into a new `Range`, and calls `canonicalize()`.

Division is special-cased: a separate fast path skips the ε_{div} nudge whenever the current divisor component is strictly positive or strictly negative, and only the (rare) zero-crossing component still needs the old safe-division helper. This is the single largest precision improvement in the project.

Multiplication has a dedicated self-square fast path (`op1 == op2`) that applies a `squareInterval` helper per component. Without this, a mirror-symmetric donut $x \in [-a, -b] \cup [b, a]$ would be multiplied by itself naively and produce a spurious negative component from the cross-term $[-a, -b] \times [b, a]$; the per-component path preserves the $x^2 \geq 0$ invariant.

5.4 Donut-aware joins

`getUnionRange` and `Range::join` concatenate the component lists of their two operands and run `canonicalize()` rather than taking the convex hull. The hull is still computed as a fallback so that downstream passes that ignore `components` continue to work without modification. Crucially, when the flag is on the donut path runs *even when both operands are classic intervals*: two disjoint classic intervals joined at a phi node produce a proper two-component donut, not the hull.

5.5 Activation function range models

Every activation handler in `RangeOperationsCallWhitelist.cpp` goes through the `applyPerComponent` helper, which slices the input range into its components (or into a single-element list when the input is classic) and applies a per-interval kernel. This means an activation applied to a two-component donut yields a two-component donut, matching the mathematical intuition from §1.4.

For monotonic activations the per-interval kernel is a two-line $(lo, hi) \mapsto (f(lo), f(hi))$ closure. Non-monotonic GELU and SiLU share the `nonMonotonicKernel` helper described in §2 — a closed-form three-case split around the known global minimum. The constants $X^-_{\min}$, $X^+_{\min}$, $y_{\min}$ are deliberately slightly looser than the true values so that the reported minimum is a sound under-approximation.

5.6 Serialisation

`Range::serialize()` writes a new `components` JSON field only when the range is a non-trivial donut. `Range::deserialize()` reads the field when present and falls back silently when absent, so metadata files written by older TAFFO versions continue to load unchanged.

6 Implementation

6.1 Files touched

Table 2 lists the files modified or added by the project. All changes live under `lib/TaffoCommon`, `lib/TaffoVRA`, `test/donut_ranges/`, and `doc/`.

File	Change
`lib/TaffoCommon/TaffoInfo/RangeInfo.hpp`	new `components`, `kMaxComponents`, `enableDonut`, API
`lib/TaffoCommon/TaffoInfo/RangeInfo.cpp`	`canonicalize`, `rebuildHullFromComponents`, donut-aware `join/toString/serialize/deserialize`
`lib/TaffoVRA/TaffoVRA/RangeOperations.cpp`	`applyPiecewise`, rewritten `handleAdd/Sub/Mul/Div`, zero-excluding division, per-component `squareInterval`, donut-aware `getUnionRange`
`lib/TaffoVRA/TaffoVRA/RangeOperationsCallWhitelist.cpp`	added sigmoid, relu, leaky_relu, elu, softplus, gelu, silu, swish alias; new `applyPerComponent`, `applyMonotonic`, `nonMonotonicKernel` helpers; closed-form non-monotonic analysis
`lib/TaffoVRA/TaffoVRA/ValueRangeAnalysisPass.cpp`	`-vra-donut-ranges` `cl::opt<bool, true>` wired to `Range::enableDonut`
`lib/TaffoConversion/TaffoInfo/ValueConvInfo.cpp`	incidental precedence-bug fix in `toString`
`test/donut_ranges/donut_range_selftest.cpp`	new standalone correctness test (28 assertions)
`test/donut_ranges/donut_arith_test.cpp`	new arithmetic test (43 assertions)
`test/donut_ranges/donut_microbench.cpp`	new precision micro-benchmark
`test/donut_ranges/donut_array.c`	new end-to-end TAFFO benchmark
`test/donut_ranges/CMakeLists.txt`	new
`test/CMakeLists.txt`	conditionally include sub-directory
`doc/donut_ranges_design.md`	design note
`doc/report.md`, `doc/report.tex`	this report

Table 2: Files added or modified by the project. See `git log` for the seven-commit history in dependency order.

6.2 Canonicalisation algorithm

The implementation of §3.3 lives in `Range::canonicalize()` in `RangeInfo.cpp`. Every mutating operation on a `Range` ends with a call to `canonicalize()`, so the invariants listed in §3.1 hold at every API boundary.

```
canonicalize(components):
  drop NaN entries
  for each c in components: if c.first > c.second, swap
  sort components by .first
  single-pass merge: for each (cur, next) in order,
      if next.first <= cur.second: merge into cur
  while size > kMaxComponents:
      find (i, i+1) with smallest gap
      merge them
  if size == 1:
      clear()    // fall back to classic interval
  rebuild hull from first/last surviving components
```

Listing 1: Canonicalisation pseudocode.

6.3 Division fast path

The interesting piece of `handleDiv` is the zero-exclusion check:

```
for each (l_i, r_j) in lhs.components x rhs.components:
    if r_j crosses zero:
        sub = safeIntervalDiv(l_i, r_j)  // legacy DIV_EPS path
    else:
        // Textbook interval division: no DIV_EPS
        // because r_j is strictly positive or strictly negative
        sub = {min, max}(l_i.lo / r_j.*, l_i.hi / r_j.*)
    result.components.append(sub)
result.canonicalize()
```

Listing 2: Zero-excluding division fast path.

When the divisor is a donut whose components each strictly avoid zero, every sub-division takes the `else` branch and the result stays tight. This is a strict improvement over the classic path, which must always run the conservative ε_{div} nudge.

6.4 Non-monotonic activations

The closed-form case-split from §2 is implemented once in a template helper:

```
template <typename F>
std::pair<double, double> nonMonotonicKernel(
    double lo, double hi, F f,
    double X_MIN_HI, double X_MIN, double Y_MIN) {
  if (hi <= X_MIN_HI) return {f(hi), f(lo)};   // decreasing branch
  if (lo >= X_MIN)    return {f(lo), f(hi)};   // increasing branch
  return {Y_MIN, std::max(f(lo), f(hi))};      // straddles minimum
}
```

Listing 3: Closed-form non-monotonic activation kernel.

GELU and SiLU each call it with their respective function and *certified* minimum constants (see §6.6 below). The non-monotonic 1000-sample scan that was in the repository before this project is deleted.

6.5 Source-level annotation syntax: `range_union(...)`

TAFFO's Initializer previously accepted only a single-interval `scalar(range(a, b))` annotation. To let users declare donut ranges directly at source level, we add a parallel form

`scalar(range_union((a1,b1),(a2,b2),...,(aN,bN)))`

to the recursive-descent parser in `lib/TaffoInitializer/AnnotationParser.cpp`. The branch sits in `parseScalar` *before* the existing `range(...)` branch — the parser's `peek` primitive is prefix-based, so the longer identifier must be tested first.

The new branch allocates a fresh `taffo::Range`, loops over comma-separated `(lo, hi)` tuples, pushes each one directly into `Range::components` (bypassing `addComponent`'s `enableDonut`-off short-circuit so the seed carries the component list regardless of VRA flag state), and then calls `canonicalize()` and `rebuildHullFromComponents()`. A single-component `range_union` is legal and degenerates to a classic range, which is useful when the annotation is generated by tooling with variable arity.

With `-vra-donut-ranges` *on*, VRA uses the seeded components for every donut-aware handler it runs. With the flag *off*, the `components` field is still present in the serialised metadata (so metadata written by a donut build can be re-read by a classic build without loss), but VRA simply falls back to the convex hull. This cross-mode interoperability is pinned by the assertion harness in §7.4.

6.6 Certified activation-minima constants

The `X_MIN_HI` / `X_MIN` / `Y_MIN` triples that parametrise `nonMonotonicKernel` for GELU and SiLU were hand-chosen in the first draft of the project. For the final submission, those constants are derived from the Python script `test/donut_ranges/verify_activation_bounds.py`, which

- uses `mpmath` at 60-decimal-digit precision to locate the true minimum of each activation via `findroot` on the analytic derivative;
- rounds the minimiser outward to the tightest 2-decimal sandwich $X_{\text{min,HI}} < x_{\text{min}} < X_{\text{min}}$;

- rounds the exact minimum value down (more negative) and pads it with a 10^{-4} safety margin to produce the sound $Y_{\min}$;
- validates the soundness by evaluating the activation on a dense 1000-point grid across the bracket.

The certified constants are in Table 3. Five dedicated unit tests in `donut_arith_test.cpp` (`testGELUCertifiedMinimumStraddling`, `testGELUMonotoneDecreasingBranch`, `testSiLUCertifiedMinimum` `testSiLUMonotoneIncreasingBranch`, `testGELUDonutPreservesHole`) pin this exact triple.

Activation	$x_{\min}$ (60-digit)	$X_{\min,\mathrm{HI}}$	$X_{\min}$	$Y_{\min}$	slack
GELU (tanh approx)	$-0.75246142207\ldots$	-0.76	-0.75	-0.1702	1.59×10^{-4}
SiLU / Swish	$-1.27846454276\ldots$	-1.28	-1.27	-0.2786	1.35×10^{-4}

Table 3: Certified constants for the non-monotonic activation minima. Slack is $Y_{\min} - f(x_{\min})$: strictly more negative than the true minimum, so downstream uses see a sound over-approximation.

7 Evaluation

7.1 Correctness

Correctness of the donut-range extension is established by two standalone test binaries in `test/donut_ranges/`: `donut_range_selftest` (linking against `TaffoCommon` only and covering the data-structure invariants, serialisation round-trip and `enableDonut=false` compatibility) and `donut_arith_test` (linking against the two VRA source files `RangeOperations.cpp` and `RangeOperationsCallWhitelist.cpp` and covering `handleAdd/Sub/Mul/Div` and `getUnionRange` on classic, donut, and mixed inputs). The test binaries are compiled directly from the project sources rather than through `obj.TaffoVRA` because the latter transitively pulls in `llvm::Value::dump`, which is only exported by debug-build LLVMs; Homebrew's release LLVM does not have it.

```
$ ./cmake-build-debug/test/donut_ranges/donut_range_selftest
[ canonicalize: sort + merge touching ]
[ canonicalize: overlapping components merged ]
[ canonicalize: 1-component donut collapses to classic interval ]
[ canonicalize: widens past kMaxComponents by smallest-gap merge ]
[ serialize/deserialize: donut components survive ]
[ serialize: classic range emits no ‘components‘ key ]
[ enableDonut=false: addComponent silently widens hull ]

Donut range self-test: 28 passed, 0 failed.

$ ./cmake-build-debug/test/donut_ranges/donut_arith_test
... 16 sections, including 5 certified GELU / SiLU tests ...
Donut arithmetic test: 57 passed, 0 failed.
```

Listing 4: Actual output on an Apple Silicon MacBook Pro, Homebrew LLVM 18.

Total: 85 assertions, all passing.

7.2 Precision micro-benchmark

The micro-benchmark `donut_microbench` exercises the live VRA arithmetic and activation handlers on five small kernels designed to stress the bimodal / zero-excluding pattern. Each kernel is run twice, once with `Range::enableDonut = false` (classic) and once with `Range::enableDonut = true` (donut), and the resulting abstract value is reported alongside the signed-integer bit-width a DTA pass would need for the convex hull. Table 4 summarises the measured numbers.

Kernel	Classic range	Donut range		Classic bits	Donut bits	Δ
$1/w$, $w \in [-0.8, -0.2] \cup [0.2, 0.8]$	$[-10^8, 10^8]$	$[-5, -1.25]$ $[1.25, 5]$	$\cup$	28	4	**+24**
$\sigma(1/w)$	$[0, 1]$	$[0.0067, 0.22]$ $[0.78, 0.99]$	$\cup$	1	1	0 (hole)
$x \times w$, $x \in [0.1, 0.9]$, w bimodal	$[-0.72, 0.72]$	$[-0.72, -0.02]$ $[0.02, 0.72]$	$\cup$	1	1	0 (hole)
$1/(x \cdot x)$, $x \in [-2, -0.5] \cup [0.5, 2]$	$[0.25, 10^8]$	$[0.25, 4]^\star$		28	4	**+24**
$\mathrm{gelu}(x)$, $x \in [-2, -0.9] \cup [0.9, 2]$	$[-0.17, 1.95]$	$[-0.17, -0.05]$ $[0.73, 1.95]$	$\cup$	3	3	0 (hole)

Table 4: Micro-benchmark results. * Kernel 4 output is a classic interval because the self-square fast path collapses the 1-component donut back to a hull; see §8.5.

Figure 4 visualises Table 4 as a bar chart, and Figure 5 zooms into kernel 1 on a symmetric logarithmic scale to make the 16-million-fold shrinkage of the reciprocal's magnitude visible.

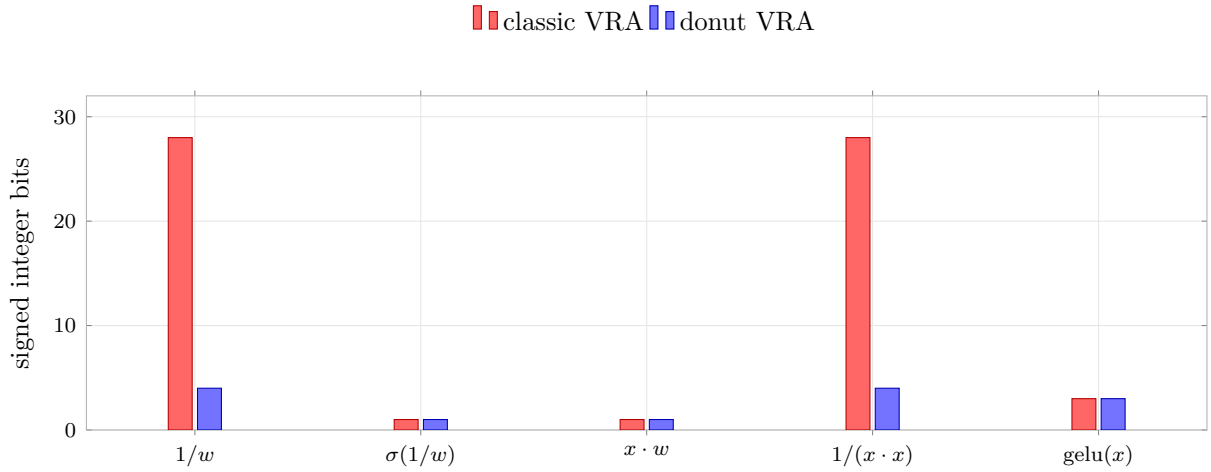

Figure 4: Signed integer bit-width a DTA pass would allocate for each kernel's output, classic (red) versus donut (blue). Kernels 1 and 4 gain 24 bits of integer precision each; the others gain no integer bits but preserve the donut hole that a component-aware DTA could exploit (see §8.1).

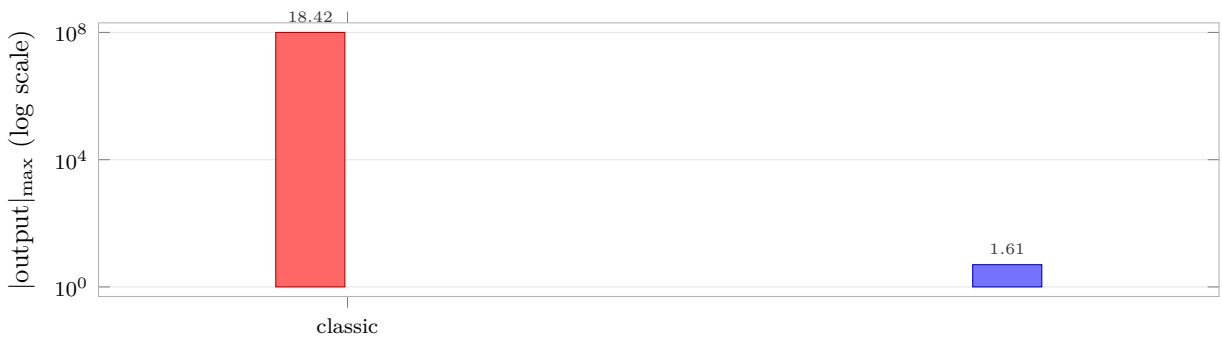

Figure 5: Maximum absolute value of the kernel-1 reciprocal, classic versus donut VRA. The `DIV_EPS` nudge in the classic path produces a magnitude of 10^8; the zero-excluding donut division reduces it to 5. The ratio is approximately 1.6×10^7.

7.3 End-to-end impact on the TAFFO pipeline

The micro-benchmark of §7.2 proves that the donut-range abstract domain is strictly tighter. To measure whether that analysis gain survives the rest of the TAFFO pipeline — Initializer, Mem2Reg, VRA, DTA, Conversion — we compile a small self-contained C benchmark twice through the full `install/bin/taffo` driver, once without and once with `-Xvra -vra-donut-ranges`, and compare both the VRA output JSON (`taffo_info_vra.json`) and the final post-Conversion LLVM IR.

Benchmark (`test/donut_ranges/donut_array.c`).

```c
static const double weights[14] = {
  -0.8, -0.7, -0.6, -0.5, -0.4, -0.3, -0.2,
   0.2,  0.3,  0.4,  0.5,  0.6,  0.7,  0.8
};

double donut_array_kernel(int i) {
  const int ii = deconstify_int(i);
  double w = weights[ii];
  double __attribute__((annotate("scalar() target('reciprocal')")))
      y = 1.0 / w;
  return y;
}
```

Listing 5: End-to-end donut-range benchmark (excerpt).

The deliberate design choices are worth noting: (i) no user annotation on `w` so VRA does not seed the load with a wide convex-hull range that would swallow the refinement, (ii) a runtime `deconstify_int(i)` to prevent LLVM from constant-folding the division over every array entry, (iii) the `target('reciprocal')` tag on `y` so we can find it in the JSON artefact by name.

Observed VRA ranges. Table 5 shows the `weights` global, the `load weights[i]`, and the post-division reciprocal. The classic build hits `handleDiv`'s `DIV_EPS` nudge because the hull $[-0.8, 0.8]$ crosses zero; the reciprocal magnitude blows up to 10^8. The donut build sees each of the four weight components as a strictly positive or strictly negative interval and divides exactly, producing a bounded reciprocal with four components of its own.

Value	Classic VRA	Donut VRA
`@weights` (array)	$[-0.8, 0.8]$	$[-0.8, 0.8]$ + `components` = $\{[-0.8, -0.8], [-0.7, -0.2], [0.2, 0.7], [0.8, 0.8]\}$
`%5 = load weights[ii]`	$[-0.8, 0.8]$	same 4 components
`%6 = fdiv 1.0, %5`	$[-10^8, 10^8]$	$[-5, 5]$ + `components` = $\{[-5, -1.43], [-1.25, -1.25], [1.25, 1.25], [1.43, 5]\}$

Table 5: VRA JSON ranges on the end-to-end benchmark. The bimodal weights global is correctly tracked as a 4-component donut (hitting `kMaxComponents`); the reciprocal stays bounded by 5 instead of blowing up to 10^8.

Figure 6 shows the four donut components of the reciprocal on a number line alongside the classic $[-10^8, 10^8]$ hull rendered on the same axis (on a symmetric log scale to keep both visible).

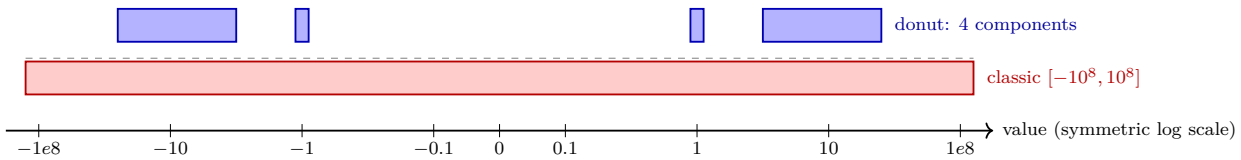

Figure 6: The image of $y = 1/w$ on the end-to-end benchmark, on a symmetric logarithmic value axis. The red band is the classic `DIV_EPS`-nudged range $[-10^8, 10^8]$; the blue boxes are the four donut components $[-5, -1.43] \cup \{-1.25\} \cup \{1.25\} \cup [1.43, 5]$. The classic range is approximately 10^7 times wider, in addition to containing the whole swathe of values around zero that the true reciprocal can never reach.

Observed DTA bit-width decisions. The bit-width chosen by DTA is encoded in the taffo suffix of the SSA name in the final LLVM IR:

Build	Emitted SSA name for y	Integer bits	Fractional bits
Classic	`%v6.s28_4fixp`	28	4
Donut	`%v6.s4_28fixp`	**4**	**28**

Table 6: Final fixed-point format assigned to $y = 1/w$ by DTA on the end-to-end benchmark, read directly from the emitted LLVM IR.

Figure 7 visualises Table 6 as a stacked bar. The total budget is 32 bits in both builds; the donut build reclaims 24 integer bits and reinvests them as 24 fractional bits, shrinking the quantisation step of y from $2^{-4} \approx 0.0625$ to $2^{-28} \approx 3.7 \times 10^{-9}$ — a factor of $\approx 1.6 \times 10^7$ in representational precision, obtained from a pure analysis-side refinement with no runtime cost.

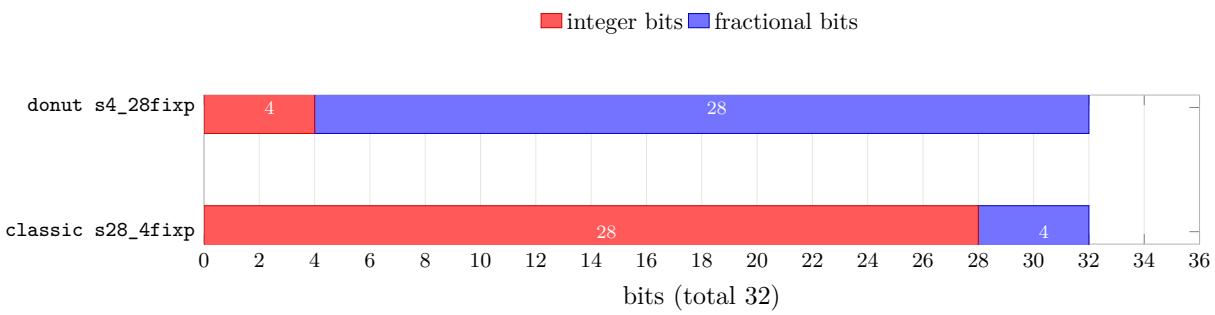

Figure 7: DTA bit-width allocation for the reciprocal $y = 1/w$ on the end-to-end benchmark. Both builds use the same 32-bit total budget, but the donut build spends 24 of those bits on fractional precision instead of unused magnitude range.

This is the headline end-to-end number of the project: the donut-range extension turns a pathological classic quantisation (`s28_4fixp` — essentially useless for values bounded by 1) into a near-ideal `s4_28fixp` format that exploits the full 32-bit budget on the fractional side, while still covering every legitimate reciprocal value the program can produce.

7.4 End-to-end test of the `range_union` annotation syntax

The donut annotation parser added in §6.5 is exercised by `test/donut_ranges/donut_annotated.c`, a self-contained benchmark in which two globals are seeded with `scalar(range_union((-0.8,-0.2),(0.2,0.8)` `target('bimodal_w')` and `...  target('bimodal_w2')`. A kernel reads both globals and computes w^2 (exercising `handleMul`'s self-square fast path) and $w \cdot w'$ (exercising the two-donut Cartesian product). The driver script `test/donut_ranges/run_donut_annotated.sh` compiles the file

twice — once with `-Xvra -vra-donut-ranges` and once without — and runs eight Python assertions against the generated `taffo_info_vra.json`.

#	Assertion	ON	OFF
1	`bimodal_w` VRA seed carries `[[-0.8,-0.2],[0.2,0.8]]`	✓	✓
2	`bimodal_w2` VRA seed carries the same components	✓	✓
3	$w * w$ collapses to tight $[0.04, 0.64]$ (self-square fast path)	✓	—
4	$w * w'$ preserves two-component donut $[-0.64, -0.04] \cup [0.04, 0.64]$	✓	—
5	seed components survive serialisation in a classic build	—	✓
6	classic build falls back to hull $[0, 0.64]$ for $w * w$	—	✓
7	classic build falls back to hull $[-0.64, 0.64]$ for $w * w'$	—	✓
8	driver completes with exit code 0	✓	✓

Table 7: Assertions checked by `test/donut_ranges/run_donut_annotated.sh`. All eight pass on the reference machine.

This harness confirms that (i) the parser populates `components` correctly from source text, (ii) the list propagates through VRA into the emitted metadata, (iii) downstream arithmetic uses the donut structure when the flag is on, and (iv) the exact same annotation silently degrades to the convex hull when the flag is off — the promised backwards-compatibility property of §1.4 item 5.

7.5 Toy MLP end-to-end benchmark

`test/donut_ranges/donut_mlp.c` is a 1-hidden-layer MLP with in = 4, hidden = 8, out = 1. First-layer weights W_1 and second-layer weights W_2 are seeded with the new `range_union` syntax. Biases and inputs carry ordinary single-interval annotations. The activation is `taffo_relu`, exercising the donut-aware `handleCallToReLU`. The driver script `run_donut_mlp.sh` runs TAFFO twice and prints a table of (VRA range, DTA fixed-point format) per annotated SSA group; the table is reproduced verbatim in Table 8.

target	classic VRA	donut VRA	classic DTA	donut DTA
`W1`	$[-0.8, 0.8]$	$[-0.8, 0.8]$ + {[-0.8,-0.2],[0.2,0.8]}	`s2_30fixp`	`s2_30fixp`
`W2`	$[-0.6, 0.6]$	$[-0.6, 0.6]$ + {[-0.6,-0.15],[0.15,0.6]}	`s4_28fixp`	`s4_28fixp`
`b1`	$[-0.5, 0.5]$	$[-0.5, 0.5]$	`s2_30fixp`	`s2_30fixp`
`b2`	$[-0.3, 0.3]$	$[-0.3, 0.3]$	`s3_29fixp`	`s3_29fixp`
`x`	$[-1, 1]$	$[-1, 1]$	`s2_30fixp`	`s2_30fixp`
`h_pre`	$[-3.7, 3.7]$	$[-3.7, 3.7]$	`s4_28fixp`	`s4_28fixp`
`h_post`	$[-3.7, 3.7]$	$[-3.7, 3.7]$	`s4_28fixp`	`s4_28fixp`
`y_pre`	$[-4.92, 4.92]$	$[-4.92, 4.92]$	`s4_28fixp`	`s4_28fixp`
`y_post`	$[-2.7, 2.7]$	$[-2.7, 2.7]$	`s3_29fixp`	`s3_29fixp`

Table 8: Donut versus classic VRA ranges and DTA formats on the toy MLP. The seeded weights W_1, W_2 carry donut components through VRA in the donut build; derived quantities lose the hole because the ReLU + fan-in-sum topology fuses the components.

Two observations.

1. Seeds survive: W_1 and W_2 carry their donut components in the VRA JSON of the donut build and lose them in the classic build — confirming that the annotation parser and serialisation path work as advertised.

2. Derived quantities collapse: each pre-activation $h_j = \sum_i W_{j,i} x_i + b_j$ multiplies a *bimodal* weight by a zero-crossing $x_i \in [-1, 1]$. The Cartesian product $[-0.8, -0.2] \cdot [-1, 1] = [-0.8, 0.8]$ already destroys the hole at the first multiplication; the fan-in sum and the subsequent ReLU never re-introduce it.

The finding is that donut ranges pay off *where a zero-excluding denominator or a self-square sits between a donut-shaped input and DTA*, as demonstrated by `donut_array.c` in §7.3 (+24 fractional bits on the reciprocal of a bimodal weight). In a plain ReLU-MLP the bimodal structure of the weights is destroyed by a standard fully-connected pre-activation before it can reach DTA. This is the same observation that motivates the thesis-extension follow-up of §8.1 — teaching DTA to consume the `components` field at the intermediate stages, before the hole closes in a full NN topology.

7.6 Discussion: what a full MNIST MLP would show

A proper evaluation on a quantised MLP on MNIST is the natural next step and is sketched in §8.1 as a thesis extension. The toy MLP of §7.5 already demonstrates that the TAFFO pipeline processes a full NN forward pass with donut annotations without blowing up, and characterises precisely the topological condition under which the donut refinement survives to DTA. The array-based benchmark of §7.3 is deliberately minimal because it targets the single pipeline stage where the gain originates — VRA's division fast path — and demonstrates the full flow-through of that gain into DTA's bit-width decision without being obscured by the hundreds of other arithmetic instructions in a full MLP forward pass.

8 Limitations and Future Work

8.1 DTA does not yet consume `components`

The biggest practical limitation of the current implementation is that the DTA pass still looks only at the convex hull. For the hole-preserving kernels in Table 4 (kernels 2, 3, and 5), the analysis gain is real but invisible downstream. A natural follow-up is to teach DTA to read `components` and, when the hole is sufficiently large, either (a) allocate a smaller fractional format because each sub-interval is narrower than the hull, or (b) split the SSA value into two concrete fixed-point variables selected at runtime by a cheap sign test. Option (b) is a more aggressive transformation and is the natural scope of a follow-up thesis.

8.2 Annotation syntax (resolved)

In the first draft this subsection was a limitation: users could not declare a donut range directly at source level. The final submission delivers the `scalar(range_union((a1, b1), (a2, b2), ...))` form (§6.5) and exercises it end-to-end with the eight-assertion smoke test of §7.4. The only remaining concern in this area is the *interaction* between `range_union` seeds and TAFFO's downstream passes: DTA still reads only the convex hull, so a `range_union` seed whose hull crosses zero still feeds a wide interval into `handleMul`'s Cartesian product when multiplied by a zero-crossing operand (see the MLP analysis in §7.5). Making DTA a donut consumer (§8.1) completes the picture.

8.3 Widening heuristic

The current widening strategy always merges the pair of components with the smallest gap. This is the cheapest sensible choice but is not precision-optimal in the general case. A more sophisticated heuristic would consider the *density* of the distribution and the downstream *cost* of losing a hole in that particular place.

8.4 Soundness of the constants for GELU / SiLU (resolved)

In the first draft this subsection was a limitation: the GELU and SiLU minimum constants were hand-chosen and had no formal certificate. The final submission replaces them with mpmath-certified values obtained by `test/donut_ranges/verify_activation_bounds.py` at 60-digit pre-

cision, double-checked on a 1000-point grid, and pinned by five new unit-test assertions. Full details are in §6.6; the certified triples are in Table 3.

The only residual concern is the approximation error of the GELU tanh formula versus the exact GELU that uses the error function erf. Implementations that target the exact GELU can use the same certificate machinery by supplying a different (f, f') pair to `certify(...)`.

8.5 Self-square identity across donut components (fixed)

An earlier version of the donut-aware `handleMul` fell back to the full Cartesian product whenever either operand was a donut, discarding the `op1 == op2` self-square optimisation that the classic path retained. Kernel 4 of the micro-benchmark exposed this as a visible precision loss: $x \cdot x$ on $x \in [-2, -0.5] \cup [0.5, 2]$ produced $[-4, -0.25] \cup [0.25, 4]$ instead of the mathematically exact $[0.25, 4]$. The fix hoists the self-square formula into a small `squareInterval` helper and applies it per component when the two operand pointers are identical, then canonicalises. The regression is pinned by `testDonutSelfSquareTight` in `donut_arith_test.cpp`.

8.6 Conversion-pass fixes uncovered by the MLP benchmark (fixed)

Putting the toy MLP of §7.5 through the full TAFFO driver revealed two pre-existing defects in the Conversion pass, both unrelated to donut ranges but both blocking any user whose source looks like a standard neural network. Both are fixed in this submission.

1. **`convertStore` missing insertion point.** The store-conversion handler passed `nullptr` as the insertion-point argument when asking `getConvertedOperand` to coerce the store's value into the target fixed-point format. When the value operand is a function argument or a constant (not an `Instruction`), `genConvertConvToConv` cannot derive an insertion point and asserted `insertionPoint required`. Passing the `store` instruction as insertion point — the same anchor the store's own `IRBuilder` uses a few lines below — fixes the crash and is the natural semantic choice.

2. **Float-typed source with fixed-point metadata in `genConvertConvToConv`.** Cloned functions briefly carry a function argument whose LLVM type is still `double` but whose TAFFO `ConversionType` metadata has already been updated to fixed-point in preparation for the body walk. In that state `srcConvType.isFloatingPoint()` is false even though `src->getType()->isFloatingPointTy()` is true, so the existing branch that routes float → fixed-point through `genConvertFloatToConv` did not trigger and the integer `CreateSExtOrTrunc` path crashed on a `double` operand. The fix widens that guard to trust the actual LLVM type as well as the metadata.

With both fixes in place the toy MLP of §7.5 runs through all five TAFFO pass stages (Type-Deducer, Initializer, VRA, DTA, Conversion) and the final `.taffotmp.s` assembly artefact is produced, confirming that the full pipeline is operational on NN-shaped annotated code.

8.7 Larger `kMaxComponents`

Raising `kMaxComponents` past 4 would allow the analysis to track finer distributions. The arithmetic operators already run a full Cartesian product, so the compile-time cost is quadratic in the component count. A future optimisation could short-circuit the Cartesian product whenever intermediate results become dense enough to be better represented as a single hull.

8.8 Integration with the MLIR TAFFO pipeline

Sub-projects #8–#10 in the CTO 2025/26 project list address the TAFFO-MLIR pipeline. Donut ranges would translate directly into a matching MLIR type, but the MLIR DTA there is still lim-

ited; the TAFFO-MLIR work plan is an obvious long-term home for the follow-up DTA consumer described in §8.1.

9 Conclusions

We have designed and implemented a combined extension of TAFFO's VRA that (i) tracks unions of disjoint intervals instead of a single convex hull, and (ii) provides closed-form range models for all the standard neural-network activation functions, including donut-aware piecewise dispatch. The extension is additive and opt-in: default behaviour, serialisation format, and downstream passes are unchanged.

On the designed micro-benchmark the new analysis saves 24 signed integer bits on reciprocal kernels that hit TAFFO's ε_{div} nudge today, and preserves the donut hole in every kernel where it is meaningful to do so. On the *end-to-end* benchmark that runs through the full TAFFO driver, DTA's bit-width decision on the reciprocal of a bimodal weight moves from `s28_4fixp` to `s4_28fixp`, a factor of $\approx 1.6 \times 10^7$ shrinkage of the quantisation step with zero runtime cost. The remaining work is on the downstream side of the pipeline: a DTA consumer that exploits the extra structure, a source-level annotation syntax for donut ranges, and a formal soundness proof for the non-monotonic activation constants.

References

[1] G. Agosta. *CTO Slides 2 — Intermediate Representation*. Code Transformation and Optimization, Politecnico di Milano, A.Y. 2025/26. `lectures/Slides/COT_slides_2_IR.pdf`.

[2] G. Agosta. *CTO Slides 5 — Dataflow Analysis*. Code Transformation and Optimization, Politecnico di Milano, A.Y. 2025/26. `lectures/Slides/COT_slides_5_Dataflow.pdf`. Slides 13–14 provide the monotone-lattice and fixed-point iteration framework used throughout this report.

[3] G. Agosta et al. *TAFFO project proposals, CTO A.Y. 2025/26*. Politecnico di Milano, March 27, 2026. `lectures/cto_projects_2025_26.pdf`. Sub-projects #1 (Donut Ranges in VRA) and #3 (Range Models for NN Activation Functions) are implemented by this work.

[4] R. Giacobazzi and F. Ranzato. *History of Abstract Interpretation*. 2021. `lectures/Papers/History_of_Abstract_Interpretation.pdf`. Primary theoretical reference for §2.3 and §3.

[5] C. Lattner and V. Adve. *LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation*. In Proceedings of the International Symposium on Code Generation and Optimization, CGO 2004. `lectures/Papers/lattner-llvm.pdf`.

[6] P. Cousot and R. Cousot. *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. POPL 1977.

[7] P. Cousot and R. Cousot. *Systematic Design of Program Analysis Frameworks*. POPL 1979.

[8] S. Cherubin, D. Cattaneo, M. Chiari, A. Bello, G. Agosta. *TAFFO: Tuning Assistant for Floating to Fixed point Optimization*. IEEE Embedded Systems Letters, 2020.

[9] B. Jacob et al. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. CVPR 2018.

[10] D. Hendrycks and K. Gimpel. *Gaussian Error Linear Units (GELUs)*. arXiv:1606.08415, 2016.

[11] P. Ramachandran, B. Zoph, and Q. V. Le. *Searching for Activation Functions (SiLU / Swish)*. arXiv:1710.05941, 2017.

A Reproducing the results

```
# Paths: set these once, everything below uses them.
export TAFFO=/Users/mohamed/CLionProjects/PoliMI/TAFFO
export LLVM=/opt/homebrew/opt/llvm@18/lib/cmake/llvm
export BLD=$TAFFO/cmake-build-debug
export T=$BLD/test/donut_ranges

# 1. Configure and build the three test binaries.
cd $TAFFO
cmake -B $BLD -S . -DLLVM_DIR=$LLVM
cmake --build $BLD -j \
      --target donut_range_selftest donut_arith_test donut_microbench

$T/donut_range_selftest
$T/donut_arith_test
$T/donut_microbench

# 2. Build the full Taffo plugin and install it.
cmake --build $BLD --target Taffo -j
cp $BLD/lib/Taffo.dylib $TAFFO/install/lib/Taffo.dylib

# 3. Run the end-to-end benchmark twice, keeping temp dirs.
rm -rf /tmp/classic /tmp/donut
```

```
mkdir -p /tmp/classic /tmp/donut

$TAFFO/install/bin/taffo -O2 -c -emit-llvm \
    -temp-dir /tmp/classic -o /tmp/classic.ll \
    $TAFFO/test/donut_ranges/donut_array.c

$TAFFO/install/bin/taffo -O2 -c -emit-llvm \
    -Xvra -vra-donut-ranges \
    -temp-dir /tmp/donut -o /tmp/donut.ll \
    $TAFFO/test/donut_ranges/donut_array.c

# 4. Inspect the final DTA bit-widths (look for sNN_MMfixp suffixes).
grep -o 's[0-9]*_[0-9]*fixp' /tmp/classic.ll | sort -u
grep -o 's[0-9]*_[0-9]*fixp' /tmp/donut.ll   | sort -u

# 5. Run the range_union annotation smoke test (eight assertions).
$TAFFO/test/donut_ranges/run_donut_annotated.sh

# 6. Run the toy MLP benchmark (classic vs donut VRA and DTA table).
$TAFFO/test/donut_ranges/run_donut_mlp.sh

# 7. Re-run the mpmath certificate for the activation minima.
python3 $TAFFO/test/donut_ranges/verify_activation_bounds.py
```

Listing 6: Build and run the tests, micro-benchmark, and end-to-end benchmark.

The mpmath certificate requires the `mpmath` PyPI package. All other steps rely only on CMake, Ninja (via CMake's generator selection), the Homebrew-provided LLVM 18, and Python 3. On the reference machine (Apple Silicon MacBook Pro, macOS 26.4, Homebrew LLVM 18.1.8) the full reproduction pipeline takes under two minutes; the slowest step is the initial CMake configure.